

مرور جامع و کاربردی کنترل کننده LQR

علیرضا قاسمی

۱۳ شهریور ۱۴۰۵

فهرست مطالب

۳	۱ خلاصه
۳	۲ LQR دقیقاً چه کاری انجام می‌دهد؟
۴	۳ شرایط و مشخصات سیستم برای استفاده از LQR
۴	۱.۳ مدل فضای حالت
۴	۲.۳ خطی بودن یا خطی‌سازی مناسب
۴	۳.۳ کنترل‌پذیری
۴	۴.۳ قابل مشاهده بودن حالت‌ها
۵	۴ تابع هزینه LQR
۵	۵ معادله ریکاتی و قانون کنترل
۶	۶ چرا LQR «بهینه» است؟
۶	۷ انتخاب ماتریس‌های Q و R
۷	۱.۷ روش Rule Bryson's
۷	۲.۷ نکته مهم درباره واحدها
۷	۸ اثر افزایش Q و R
۷	۹ LQR در برابر Placement Pole
۸	۱۰ محدودیت‌ها و نقاط ضعف LQR
۸	۱.۱۰ عدم لحاظ مستقیم قیود
۸	۲.۱۰ نیاز به فیدبک حالت
۸	۳.۱۰ وابستگی به مدل
۸	۴.۱۰ مناسب نبودن برای غیرخطی‌های شدید
۹	۵.۱۰ مسئله tracking به صورت پیش‌فرض
۹	۱۱ LQR برای Tracking
۹	۱۲ LQI: افزودن عمل انتگرالی به LQR
۱۰	۱۳ حالت گسسته LQR
۱۰	۱۴ پیاده‌سازی LQR در MATLAB
۱۱	۱۵ بررسی کنترل‌پذیری قبل از LQR
۱۱	۱۶ استفاده از CARE در MATLAB

۱۲	۱۷: MATLAB یک روال طراحی پیشنهادی
۱۲	۱۸ یک مثال مهندسی: موتور DC
۱۳	۱۹ اشباع عملگر؛ مهم‌ترین نکته عملی
۱۴	۲۰ نویز اندازه‌گیری
۱۴	۲۱ LQG
۱۴	۲۲ مقایسه LQR با PID
۱۵	۲۳ LQR و MPC
۱۵	۲۴ LQR و Control Robust
۱۶	۲۵ Cross Weighting: ماتریس N
۱۶	۲۶ LQR Finite-Horizon
۱۶	۲۷ TVLQR و سیستم‌های غیرخطی
۱۷	۲۸ LQR و مقیاس‌بندی حالت‌ها
۱۷	۲۹ نکات مهم برای طراحی حرفه‌ای
۱۸	۳۰ اشتباهات رایج
۱۸	۱.۳۰ اشتباه ۱: تصور اینکه LQR همیشه بهترین کنترل‌کننده است
۱۸	۲.۳۰ اشتباه ۲: انتخاب $Q=I$ و $R=1$ بدون تحلیل
۱۸	۳.۳۰ اشتباه ۳: نادیده گرفتن اشباع
۱۸	۴.۳۰ اشتباه ۴: استفاده مستقیم از LQR روی سیستم غیرخطی بزرگ‌دامنه
۱۸	۵.۳۰ اشتباه ۵: فرض اینکه LQR مشکل error steady-state را خودکار حل می‌کند
۱۸	۶.۳۰ اشتباه ۶: اندازه‌گیری نکردن همه حالت‌ها
۱۸	۷.۳۰ اشتباه ۷: بررسی نکردن نویز و تأخیر
۱۸	۳۱ چک‌لیست طراحی LQR
۱۹	۳۲ فرمول‌های اصلی در یک نگاه
۱۹	۳۳ توابع مهم MATLAB
۲۰	۳۴ یک دید مهندسی برای انتخاب روش کنترل
۲۰	۳۵ جمع‌بندی نهایی

۱ خلاصه

کنترل‌کننده LQR یا Linear Quadratic Regulator یکی از مهم‌ترین روش‌های کنترل بهینه برای سیستم‌های خطی است. ایده اصلی آن این است که به‌جای تعیین مستقیم قطب‌ها یا تنظیم جداگانه ضرایب فیدبک، یک تابع هزینه تعریف کنیم که هم «بزرگی خطای حالت» و هم «مصرف عملگر» را جریمه کند. سپس بهره فیدبک حالت به‌گونه‌ای محاسبه می‌شود که این هزینه کمینه شود. برای سیستم

$$\dot{x} = Ax + Bu,$$

قانون کنترل استاندارد LQR به صورت

$$u = -Kx$$

است که در آن ماتریس بهره K از حل معادله جبری ریکاتی به دست می‌آید:

$$A^T P + PA - PBR^{-1}B^T P + Q = 0,$$

9

$$K = R^{-1}B^T P.$$

LQR زمانی بسیار جذاب است که:

- مدل خطی یا خطی‌شده مناسبی از سیستم داشته باشیم؛
- متغیرهای حالت در دسترس باشند یا بتوان آنها را با ناظر تخمین زد؛
- بخواهیم بین عملکرد و انرژی/مقدار فرمان مصالحه ایجاد کنیم؛
- چند متغیر حالت و چند ورودی به صورت هم‌زمان در طراحی دخیل باشند.

نکته بسیار مهم این است که LQR ذاتاً یک کنترل‌کننده constrained نیست؛ یعنی محدودیت‌هایی مانند اشباع عملگر، حد سرعت موتور، حد جریان یا حد ولتاژ مستقیماً در مسئله استاندارد LQR لحاظ نمی‌شوند.

۲ LQR دقیقاً چه کاری انجام می‌دهد؟

فرض کنید یک سیستم خطی با مدل حالت داریم:

$$\dot{x}(t) = Ax(t) + Bu(t),$$

LQR به دنبال قانون کنترلی از نوع فیدبک حالت است:

$$u(t) = -Kx(t)$$

که باعث می‌شود سیستم ضمن رسیدن به عملکرد مطلوب، یک معیار هزینه مشخص را کمینه کند. اگر Q بزرگ انتخاب شود، انحراف حالت‌ها از صفر شدیدتر جریمه می‌شود و کنترل‌کننده معمولاً برای برگرداندن حالت‌ها به صفر فرمان قوی‌تری تولید می‌کند. اگر R بزرگ انتخاب شود، استفاده از عملگر بیشتر جریمه می‌شود و کنترل‌کننده محافظه‌کارتر خواهد شد. بنابراین می‌توان LQR را به صورت یک ابزار طراحی برای ایجاد مصالحه بین

عملکرد دینامیکی

و

هزینه/تلاش کنترلی

در نظر گرفت.

۳ شرایط و مشخصات سیستم برای استفاده از LQR

مهم‌ترین شرایط عبارت‌اند از:

۱.۳ مدل فضای حالت

باید ماتریس‌های A و B را داشته باشیم.
برای سیستم گسسته:

$$x_{k+1} = A_d x_k + B_d u_k.$$

۲.۳ خطی بودن یا خطی‌سازی مناسب

LQR کلاسیک برای سیستم خطی طراحی شده است. اگر سیستم غیرخطی باشد، می‌توان آن را حول یک نقطه کار خطی کرد:

$$\dot{x} = f(x, u)$$

و در نقطه کار (x_0, u_0) تقریب زد:

$$\delta \dot{x} = A \delta x + B \delta u.$$

سپس LQR روی مدل خطی‌شده اعمال می‌شود.
برای سیستم‌های غیرخطی در گستره کاری وسیع، یک LQR ثابت ممکن است عملکرد مناسبی نداشته باشد. در این شرایط روش‌هایی مانند iLQR، TVLQR، gain scheduling یا کنترل بهینه غیرخطی می‌توانند مناسب‌تر باشند.

۳.۳ کنترل‌پذیری

برای طراحی فیدبک حالت، سیستم باید حداقل دارای ویژگی stabilizability باشد. شرط قوی‌تر و رایج‌تر، کنترل‌پذیری کامل است:

$$\mathcal{C} = [B \quad AB \quad A^2B \quad \dots \quad A^{n-1}B]$$

9

$$\text{rank}(\mathcal{C}) = n.$$

برای وجود یک جواب پایدارکننده (stabilizing solution) برای CARE، کنترل‌پذیری کامل زوج (A, B) الزاماً لازم نیست. کافی است زوج (A, B) پایدارپذیر (stabilizable) باشد؛ یعنی تمام مدهای ناپایدار سیستم توسط ورودی قابل کنترل باشند. البته برای تضمین وجود جواب stabilizing در مسئله استاندارد LQR، علاوه بر stabilizability، شرایط مناسبی مانند $Q \succeq 0$ ، $R \succ 0$ و detectability زوج $(Q^{1/2}, A)$ نیز در نظر گرفته می‌شوند.

۴.۳ قابل مشاهده بودن حالت‌ها

LQR استاندارد فرض می‌کند x را داریم. اگر همه حالت‌ها اندازه‌گیری نشوند، باید از یک ناظر مانند Luenberger observer یا Kalman filter استفاده کنیم.
این ترکیب به LQG منجر می‌شود.

۴ تابع هزینه LQR

برای LQR پیوسته با افق بی‌نهایت، تابع هزینه معمولاً به صورت زیر تعریف می‌شود:

$$J = \int_0^{\infty} (x^T Q x + u^T R u + 2x^T N u) dt.$$

در حالت استاندارد و بدون جمله متقابل:

$$J = \int_0^{\infty} (x^T Q x + u^T R u) dt$$

که در آن:

$$Q = Q^T \succeq 0$$

9

$$R = R^T \succ 0.$$

مفهوم هر جمله:

$$x^T Q x$$

هزینه مربوط به انحراف حالت‌ها است و

$$u^T R u$$

هزینه مربوط به تلاش کنترلی است.
اگر

$$Q = \begin{bmatrix} q_1 & 0 \\ 0 & q_2 \end{bmatrix}$$

باشد، افزایش q_1 باعث می‌شود کنترل‌کننده نسبت به حالت اول حساس‌تر شود.
اگر R افزایش یابد، استفاده از ورودی بیشتر جریمه می‌شود.

۵ معادله ریکاتی و قانون کنترل

مسئله LQR با استفاده از اصل بهینگی بلمن به معادله ریکاتی منتهی می‌شود.
برای مسئله پیوسته با افق بی‌نهایت، معادله جبری ریکاتی:

$$A^T P + P A - P B R^{-1} B^T P + Q = 0$$

است.

پس از محاسبه P ، بهره LQR برابر است با:

$$K = R^{-1} B^T P$$

و قانون کنترل:

$$u = -Kx.$$

با جایگذاری قانون کنترل در مدل:

$$\dot{x} = Ax - BKx$$

بنابراین:

$$\dot{x} = (A - BK)x$$

و ماتریس حلقه بسته:

$$A_{cl} = A - BK.$$

برای بررسی پایداری، مقادیر ویژه

$$\lambda(A - BK)$$

را بررسی می‌کنیم.

اگر تمام قسمت‌های حقیقی مقادیر ویژه منفی باشند، سیستم پیوسته حلقه بسته پایدار است.

Finite- و Infinite-Horizon LQR در کنترل کننده LQR، انتخاب افق زمانی به ماهیت مسئله بستگی دارد و الزام عمومی برای استفاده از Finite-Horizon LQR وجود ندارد. در حالت Finite-Horizon، افق زمانی مشخصی با N مرحله در نظر گرفته می‌شود و قانون کنترل به صورت $u_k = -K_k x_k$ بوده و به دلیل وجود شرایط پایانی، معمولاً K_k در طول افق تغییر می‌کند؛ این روش برای مسائلی مناسب است که زمان پایان مشخص یا شرایط پایانی مهمی دارند. در مقابل، در Infinite-Horizon LQR تابع هزینه تا بی‌نهایت در نظر گرفته می‌شود و در شرایط مناسب، یک بهره ثابت K به دست می‌آید که قانون کنترل $u_k = -K x_k$ را در تمام مراحل اعمال می‌کند؛ این روش برای سیستم‌های دائمی و مسائل تنظیم و پایداری معمول، مانند بسیاری از کاربردهای صنعتی، مناسب‌تر است و در متلب با دستور `dlqr` قابل محاسبه است. لازم است توجه شود که N در Finite-Horizon LQR یک پارامتر طراحی و بخشی از مسئله بهینه‌سازی است، اما نباید آن را با معیارهایی مانند زمان صعود اشتباه گرفت؛ Rise Time یک مشخصه عملکردی سیستم است که پس از طراحی کنترل کننده بررسی می‌شود. بنابراین، برای یک سیستم دائمی می‌توان از Infinite-Horizon LQR استفاده کرد و با تنظیم Q و R ، عملکردهایی مانند زمان صعود، زمان نشست و تلاش کنترلی را به مشخصات مورد نظر نزدیک کرد. همچنین، در بسیاری از سیستم‌های مناسب، با افزایش N در Finite-Horizon LQR، بهره اولیه K_0 به بهره Infinite-Horizon همگرا می‌شود.

۶ چرا LQR «بهینه» است؟

بهینه بودن LQR به این معنی نیست که کنترل کننده در همه معیارهای ممکن بهترین است. بلکه به این معنی است که برای مدل، قیود و تابع هزینه تعریف شده، قانون کنترل انتخاب شده مقدار J را کمینه می‌کند. بنابراین اگر تابع هزینه را تغییر دهیم، کنترل کننده بهینه نیز تغییر خواهد کرد. برای مثال:

$$J_1 = \int (x^T Q x + u^T R u) dt$$

9

$$J_2 = \int (x^T Q_2 x + u^T R_2 u) dt$$

دو مسئله متفاوت هستند و به طور کلی $K_1 \neq K_2$. این نکته در تنظیم LQR بسیار مهم است:

در واقع مشخص کننده فلسفه طراحی کنترل کننده هستند Q و R

۷ انتخاب ماتریس‌های Q و R

یکی از مهم‌ترین قسمت‌های طراحی LQR، انتخاب Q و R است. یک انتخاب ساده:

$$Q = \text{diag}(q_1, \dots, q_n)$$

$$R = \text{diag}(r_1, \dots, r_m).$$

اما انتخاب اعداد کاملاً دلخواه معمولاً روش خوبی نیست.

۱.۷ روش Rule Bryson's

یکی از روش‌های عملی برای شروع:

$$Q_{ii} = \frac{1}{x_{i,\max}^2}$$

$$R_{jj} = \frac{1}{u_{j,\max}^2}.$$

در نتیجه:

$$J \approx \int \left[\sum_i \left(\frac{x_i}{x_{i,\max}} \right)^2 + \sum_j \left(\frac{u_j}{u_{j,\max}} \right)^2 \right] dt.$$

این روش باعث می‌شود متغیرها بر اساس مقیاس فیزیکی مناسب‌تری وزن‌دهی شوند.

۲.۷ نکته مهم درباره واحدها

اگر یک حالت بر حسب متر و حالت دیگر بر حسب رادیان یا آمپر باشد، استفاده از مقادیر عددی خام ممکن است وزن‌دهی نامعقولی ایجاد کند. بنابراین نرمال‌سازی یا استفاده از مقیاس‌های فیزیکی قابل قبول برای حالت‌ها و ورودی‌ها اهمیت زیادی دارد.

۸ اثر افزایش Q و R

به صورت کیفی:

پارامتر	افزایش آن	نتیجه معمول
Q	بیشتر	حالت‌ها شدیدتر جریمه می‌شوند
R	بیشتر	تلاش کنترلی بیشتر جریمه می‌شود
Q/R	بیشتر	کنترل تهاجمی‌تر
Q/R	کمتر	کنترل محافظه‌کارانه‌تر

اما این رابطه را نباید به صورت یک قانون مطلق برای تمام مشخصات زمانی تفسیر کرد؛ تغییر Q و R می‌تواند همه قطب‌های حلقه بسته و پاسخ سیستم را به شکل کوپل شده تغییر دهد.

۹ LQR در برابر Pole Placement

در Pole Placement شما مستقیماً هدف قرار می‌دهید که قطب‌های حلقه بسته کجا قرار بگیرند:

$$\det(sI - A + BK) = 0.$$

در LQR، شما مستقیماً قطب‌ها را انتخاب نمی‌کنید؛ بلکه تابع هزینه را تعریف می‌کنید.

ویژگی	LQR	Placement Pole
هدف طراحی	کمینه سازی هزینه	قرار دادن قطبها
تنظیم عملکرد MIMO	از طریق Q, R	از طریق قطبها
	بسیار مناسب	مناسب ولی حساس تر به انتخاب قطبها
مفهوم بهینه سازی	دارد	ندارد
مصرف کنترل	مستقیماً در R لحاظ می شود	مستقیماً لحاظ نمی شود

مزیت مهم LQR این است که تلاش کنترلی را مستقیماً در مسئله طراحی وارد می کند.

۱۰ محدودیت ها و نقاط ضعف LQR

۱.۱۰ ۱. عدم لحاظ مستقیم قیود

LQR استاندارد محدودیت هایی مانند

$$u_{\min} \leq u \leq u_{\max}$$

را مستقیماً در مسئله وارد نمی کند. این یک محدودیت بسیار مهم در سیستم های صنعتی است. اگر کنترل کننده فرمانی بزرگ تولید کند ولی عملگر اشباع شود:

$$u_{\text{actual}} \neq u_{\text{LQR}},$$

بنابراین دینامیک واقعی دیگر همان دینامیک فرض شده در طراحی نیست. برای مسائل constrained معمولاً MPC گزینه طبیعی تری است.

۲.۱۰ ۲. نیاز به فیدبک حالت

اگر همه حالت ها اندازه گیری نشوند، نمی توان مستقیماً $u = -Kx$ را اجرا کرد. راه حل:

$$u = -K\hat{x}$$

که در آن $\hat{x}$ توسط observer تخمین زده می شود.

۳.۱۰ ۳. وابستگی به مدل

اگر مدل

$$(A, B)$$

با سیستم واقعی اختلاف زیادی داشته باشد، عملکرد ممکن است افت کند. LQR کلاسیک ذاتاً یک روش control robust به معنای دقیق کلمه نیست. برای عدم قطعیت شدید، روش هایی مانند H-infinity یا روش های control robust مناسب تر هستند.

۴.۱۰ ۴. مناسب نبودن برای غیرخطی های شدید

یک LQR ثابت فقط در محدوده ای که مدل خطی مناسب است معمولاً عملکرد خوبی دارد. برای سیستم غیرخطی بزرگ دامنه ممکن است به:

• Scheduling Gain

• control LPV

- TVLQR
- iLQR
- MPC nonlinear

نیاز داشته باشیم.

۵.۱۰. مسئله tracking به صورت پیش فرض

فرم پایه: LQR

$$u = -Kx$$

برای regulation است، یعنی هدف معمولاً:

$$x \rightarrow 0.$$

اگر هدف tracking باشد:

$$r(t) \rightarrow y(t),$$

باید ساختار مناسبتری استفاده کنیم.

۱۱ LQR برای Tracking

یکی از روش‌های ساده برای tracking، استفاده از پیش‌فیلتر مرجع است:

$$u = -Kx + Nr.$$

اگر

$$y = Cx + Du$$

باشد، N می‌تواند برای ایجاد بهره استاتیکی مناسب انتخاب شود. اما در سیستم‌های واقعی، این روش همیشه به تنهایی کافی نیست؛ مخصوصاً وقتی مدل خطا داشته باشد یا اغتشاش ثابت وجود داشته باشد. روش بسیار رایج‌تر، افزودن انتگرال خطا است.

۱۲ LQI: افزودن عمل انتگرالی به LQR

خطا را تعریف کنید:

$$e = r - y.$$

و یک حالت انتگرالی جدید:

$$\dot{x}_I = e = r - y.$$

سپس حالت توسعه‌یافته:

$$x_a = \begin{bmatrix} x \\ x_I \end{bmatrix}$$

را تشکیل می‌دهیم.
کنترل‌کننده:

$$u = -K_x x - K_I x_I.$$

این روش معمولاً با عنوان LQI شناخته می‌شود. مزیت مهم LQI نسبت به LQR ساده این است که می‌تواند خطای ماندگار ناشی از مدل ناقص یا اغتشاش ثابت را کاهش دهد. اما اضافه کردن انتگرال نیز می‌تواند مسائل جدیدی ایجاد کند:

- افزایش مرتبه سیستم؛
- احتمال windup در حضور اشباع؛
- نیاز به تنظیم وزن حالت انتگرالی؛
- امکان کاهش حاشیه پایداری در صورت طراحی نامناسب.

۱۳ حالت گسسته LQR

برای سیستم گسسته:

$$x_{k+1} = A_d x_k + B_d u_k$$

تابع هزینه:

$$J = \sum_{k=0}^{\infty} (x_k^T Q x_k + u_k^T R u_k).$$

معادله ریکاتی گسسته:

$$P = A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A + Q$$

و بهره:

$$K = (R + B^T P B)^{-1} B^T P A$$

قانون کنترل:

$$u_k = -K x_k.$$

در متلب:

$$[K, S, e] = dlqr(A, B, Q, R);$$

۱۴ پیاده‌سازی LQR در MATLAB

فرض کنید:

$$A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

در MATLAB:

```
A = [0 1;
     -2 -3];

B = [0;
     1];

Q = diag([10 1]);
R = 1;

[K,S,e] = lqr(A,B,Q,R);
```

در این خروجی:

• K : بهره LQR؛

• S : ماتریس P حل معادله ریکاتی؛

• e : مقادیر ویژه حلقه بسته.

سپس:

```
Acl = A - B*K;
eig(Acl)
```

برای شبیه سازی:

```
sys_cl = ss(A-B*K, B, eye(size(A)), zeros(size(B)));
initial(sys_cl, x0)
grid on
```

۱۵ بررسی کنترل پذیری قبل از LQR

قبل از طراحی:

```
Co = ctrb(A,B);
rank(Co)
size(A,1)
```

اگر:

```
rank(Co) == size(A,1)
```

باشد، سیستم کاملاً کنترل پذیر است.

برای بررسی دقیق تر stabilizability در مسائل حرفه ای، صرفاً rank کنترل پذیری کامل را معیار نهایی قرار ندهید؛ شرط نظری مورد نیاز برای CARE، پایدارپذیری زوج (A, B) و شرایط مناسب مربوط به Q است.

۱۶ استفاده از CARE در MATLAB

می توان مسئله را مستقیماً با تابع care نیز حل کرد.
برای حالت استاندارد:

```
[P,~,K] = care(A,B,Q,R);
```

در بسیاری از نسخه های MATLAB، خروجی ها و syntax دقیق care باید با مستندات همان نسخه بررسی شود، به خصوص وقتی جمله cross-weighting N وجود دارد.
برای طراحی عملی، lqr معمولاً انتخاب ساده تر است:

```
[K,S,e] = lqr(A,B,Q,R);
```

۱۷ MATLAB: یک روال طراحی پیشنهادی

یک workflow مناسب:

```
% 1) Define model
A = ...;
B = ...;
C = ...;
D = ...;

% 2) Check dimensions
n = size(A,1);
m = size(B,2);

% 3) Check controllability
Co = ctrb(A,B);
fprintf('Controllability_rank=%d/%d\n',rank(Co),n);

% 4) Choose Q and R
Q = diag([...]);
R = diag([...]);

% 5) LQR design
[K,S,e] = lqr(A,B,Q,R);

% 6) Closed-loop model
Acl = A-B*K;
sys_cl = ss(Acl,B,C,D);

% 7) Check closed-loop poles
disp(eig(Acl));

% 8) Simulate
x0 = [...];
initial(sys_cl,x0);
grid on;
```

۱۸ یک مثال مهندسی: موتور DC

یک مدل ساده موتور DC را در نظر بگیرید:

$$J\dot{\omega} + b\omega = Ki,$$

$$L\dot{i} + Ri = V - K\omega.$$

حالت‌ها را انتخاب می‌کنیم:

$$x = \begin{bmatrix} \omega \\ i \end{bmatrix}.$$

پس:

$$\dot{\omega} = -\frac{b}{J}\omega + \frac{K}{J}i$$

9

$$\dot{i} = -\frac{K}{L}\omega - \frac{R}{L}i + \frac{1}{L}V.$$

بنابراین:

$$A = \begin{bmatrix} -\frac{b}{J} & \frac{K}{J} \\ -\frac{K}{L} & -\frac{R}{L} \end{bmatrix},$$
$$B = \begin{bmatrix} 0 \\ \frac{1}{L} \end{bmatrix}.$$

سپس می‌توان نوشت:

$$u = V = -K_{\text{LQR}}x.$$

در عمل باید بررسی شود که:

$$|V| \leq V_{\text{max}}.$$

اگر LQR ولتاژ بیشتری تولید کند، عملگر اشباع می‌شود و طراحی ایده‌آل دیگر دقیقاً برقرار نیست. این مثال نشان می‌دهد که در یک پروژه صنعتی واقعی، محاسبه K فقط مرحله اول طراحی است؛ بررسی اشباع، سنسورها، نویز، تأخیر، نمونه‌برداری و مدل واقعی نیز ضروری است.

۱۹ اشباع عملگر؛ مهم‌ترین نکته عملی

فرض کنید:

$$u = -Kx$$

اما عملگر فقط می‌تواند:

$$u_{\min} \leq u \leq u_{\max}$$

را تولید کند.

در این حالت:

$$u_{\text{actual}} = \text{sat}(-Kx).$$

این موضوع می‌تواند باعث شود:

- زمان نشست افزایش یابد؛
 - overshoot ایجاد شود؛
 - عملکرد پیش‌بینی‌شده از بین برود؛
 - در سیستم‌های دارای انتگرال، windup رخ دهد؛
 - حتی در شرایط شدید، پایداری عملی کاهش یابد.
- پس برای سیستم صنعتی، صرفاً گزارش K کافی نیست. باید trajectory فرمان را نیز بررسی کنید.

۲۰ نویز اندازه‌گیری

اگر:

$$u = -Kx$$

و اندازه‌گیری‌ها noisy باشند، نویز می‌تواند وارد ورودی کنترل شود. به همین دلیل، در سیستم‌هایی که حالت‌ها مستقیماً قابل اندازه‌گیری نیستند، از estimator استفاده می‌شود:

$$u = -K\hat{x}.$$

اگر estimator بر اساس مدل نویز فرایند و نویز اندازه‌گیری با filter Kalman طراحی شود، ساختار LQG شکل می‌گیرد.

۲۱ LQG

LQG ترکیب:

LQR + Filter Kalman

است.

LQR مسئله کنترل را حل می‌کند:

$$u = -K\hat{x}.$$

filter Kalman تخمین حالت را انجام می‌دهد:

$$\hat{x} \leftarrow \text{model} + \text{measurements}.$$

این ساختار برای سیستم‌هایی که تمام حالت‌ها قابل اندازه‌گیری نیستند بسیار مهم است. با این حال، نباید LQG را به‌طور خودکار معادل «robust control» دانست. حساسیت LQG به mismatch مدل و عدم قطعیت همچنان می‌تواند مسئله‌ساز باشد.

۲۲ مقایسه LQR با PID

PID:

$$u = K_p e + K_i \int e dt + K_d \frac{de}{dt}$$

معمولاً برای ساختارهای ساده‌تر و مسائل SISO بسیار کاربردی است.
LQR:

$$u = -Kx$$

از مدل فضای حالت و همه حالت‌ها استفاده می‌کند.
LQR زمانی مزیت بیشتری دارد که:

- سیستم MIMO باشد؛
- حالت‌ها دارای coupling باشند؛
- دینامیک چند مرتبه‌ای باشد؛
- بخواهیم effort کنترلی را نیز وارد طراحی کنیم؛

- مدل state-space مناسبی داشته باشیم.
- PID زمانی ممکن است انتخاب صنعتی ساده تر و مناسب تری باشد که:
- سیستم SISO باشد؛
- محدودیت های سخت و ساختار ساده داشته باشیم؛
- مدل دقیق فضای حالت در دسترس نباشد؛
- commissioning و تنظیم میدانی اهمیت زیادی داشته باشد.

۲۳ MPC و LQR

یکی از مهم ترین تفاوت ها:

بهینه سازی بدون قیود در حالت استاندارد $LQR \rightarrow$

در حالی که:

بهینه سازی همراه با قیود و پیش بینی آینده $MPC \rightarrow$

است.

اگر محدودیت های زیر مهم باشند:

$$u_{\min} \leq u \leq u_{\max}$$

9

$$x_{\min} \leq x \leq x_{\max},$$

MPC معمولاً انتخاب طبیعی تری است.

جالب است بدانید که LQR را می توان در حالت خاصی به عنوان یک مسئله بهینه سازی افق بی نهایت بدون قید در نظر گرفت.

۲۴ LQR و Control Robust

LQR عملکرد بسیار خوبی روی مدل نامی می تواند داشته باشد، اما تضمین performance robust در برابر عدم قطعیت شدید را به شکل مستقیم هدف قرار نمی دهد. اگر مدل به صورت:

$$A = A_0 + \Delta A$$

باشد، عملکرد LQR به ساختار و اندازه ΔA وابسته خواهد بود. برای مسائل robustness می توان روش هایی مانند:

• control H_{∞}

• μ -synthesis

• MPC robust

• shaping loop

را بررسی کرد.

۲۵ Weighting: Cross ماتریس N

فرم عمومی‌تر تابع هزینه:

$$J = \int_0^\infty (x^T Q x + 2x^T N u + u^T R u) dt.$$

در این حالت جمله N تعامل مستقیم بین حالت و ورودی را مدل می‌کند.
اگر:

$$N = 0,$$

به فرم استاندارد می‌رسیم.
در MATLAB می‌توان در طراحی‌های پیشرفته از فرم‌های دارای N استفاده کرد؛ syntax دقیق تابع را باید با نسخه MATLAB بررسی کرد.
نکته مهم این است که برای مثبت معین بودن هزینه، معمولاً شرط بلوکی زیر مطرح می‌شود:

$$\begin{bmatrix} Q & N \\ N^T & R \end{bmatrix} \succeq 0.$$

۲۶ LQR Finite-Horizon

LQR کلاسیک فوق مسئله افق بی‌نهایت است.
در LQR: finite-horizon

$$J = x(T)^T F_T x(T) + \int_0^T (x^T Q x + u^T R u) dt.$$

در این حالت $P(t)$ دیگر ثابت نیست و از معادله دیفرانسیل ریکاتی به دست می‌آید:

$$-\dot{P} = A^T P + P A - P B R^{-1} B^T P + Q.$$

شرط نهایی:

$$P(T) = F_T.$$

بهره نیز زمان متغیر می‌شود:

$$K(t) = R^{-1} B^T P(t).$$

بنابراین:

$$u(t) = -K(t)x(t).$$

این ایده در مسائل tracking trajectory و کنترل بهینه بسیار مهم است.

۲۷ TVLQR و سیستم‌های غیرخطی

اگر سیستم حول یک trajectory حرکت کند، به جای یک نقطه تعادل ثابت، می‌توان در طول trajectory خطی‌سازی کرد:

$$\delta \dot{x} = A(t)\delta x + B(t)\delta u.$$

سپس TVLQR:

$$\delta u = -K(t)\delta x$$

را طراحی می‌کند.
این روش در رباتیک و aerospace بسیار مهم است.

۲۸ LQR و مقیاس‌بندی حالت‌ها

یکی از اشتباهات رایج این است که بدون توجه به واحدها:

$$Q = I$$

انتخاب شود.
فرض کنید:

$$x_1 = 0.001 \text{ m}$$

9

$$x_2 = 100 \text{ rad/s.}$$

اگر هر دو را با وزن یکسان وارد کنیم، حالت دوم از نظر عددی بسیار بزرگ‌تر است و ممکن است اثر آن بر تابع هزینه بسیار متفاوت باشد.
راهکار:

```
Sx = diag([x1_scale, x2_scale, ...]);  
Su = diag([u1_scale, u2_scale, ...]);
```

یا استفاده از وزن‌های مبتنی بر حدود فیزیکی.

۲۹ نکات مهم برای طراحی حرفه‌ای

۱. قبل از LQR، مدل را اعتبارسنجی کنید.
۲. controllability یا حداقل stabilizability را بررسی کنید.
۳. واحدها و مقیاس متغیرهای حالت را بررسی کنید.
۴. Q و R را بر اساس اهمیت فیزیکی و محدودیت‌های واقعی انتخاب کنید.
۵. ابتدا از Rule Bryson's به عنوان نقطه شروع استفاده کنید.
۶. فقط به مقدار K نگاه نکنید؛ قطب‌های حلقه‌بسته را بررسی کنید.
۷. پاسخ زمانی، overshoot و time settling را بررسی کنید.
۸. فرمان عملگر را بررسی کنید.
۹. اشباع عملگر را در شبیه‌سازی وارد کنید.
۱۰. نویز سنسور را وارد تست کنید.
۱۱. عدم قطعیت پارامترهای مدل را تست کنید.
۱۲. اگر tracking دارید، از ساختار مناسب tracking/LQI استفاده کنید.
۱۳. اگر حالت‌ها اندازه‌گیری نمی‌شوند، observer طراحی کنید.
۱۴. اگر محدودیت‌ها حیاتی هستند، LQR را با MPC مقایسه کنید.
۱۵. اگر nonlinearities شدید هستند، فقط به یک LQR ثابت تکیه نکنید.

۳۰ اشتباهات رایج

۱.۳۰ اشتباه ۱: تصور اینکه LQR همیشه بهترین کنترل کننده است

LQR فقط برای مسئله هزینه‌ای که تعریف کرده‌اید بهینه است.

۲.۳۰ اشتباه ۲: انتخاب $Q=I$ و $R=1$ بدون تحلیل

این انتخاب می‌تواند برای یک مثال آموزشی مناسب باشد، اما در پروژه صنعتی الزاماً انتخاب خوبی نیست.

۳.۳۰ اشتباه ۳: نادیده گرفتن اشباع

این یکی از جدی‌ترین خطاهای عملی است.

۴.۳۰ اشتباه ۴: استفاده مستقیم از LQR روی سیستم غیرخطی بزرگ دامنه

LQR محلی است، مگر اینکه ساختارهای دیگری مانند scheduling gain یا TVLQR استفاده شوند.

۵.۳۰ اشتباه ۵: فرض اینکه LQR مشکل $\text{error steady-state}$ را خودکار حل می‌کند

LQR استاندارد الزاماً خطای ماندگار صفر برای tracking ایجاد نمی‌کند.

۶.۳۰ اشتباه ۶: اندازه‌گیری نکردن همه حالت‌ها

اگر x در دسترس نیست، باید estimator در نظر گرفته شود.

۷.۳۰ اشتباه ۷: بررسی نکردن نویز و تأخیر

یک K مناسب روی مدل بدون تأخیر ممکن است در سیستم واقعی با تأخیر عملکرد ضعیفی داشته باشد.

۳۱ چک‌لیست طراحی LQR

۱. مدل A, B, C, D را آماده کن.

۲. نقطه کار را مشخص کن.

۳. در صورت نیاز سیستم را خطی کن.

۴. کنترل‌پذیری/stabilizability را بررسی کن.

۵. حالت‌های قابل اندازه‌گیری را مشخص کن.

۶. حدود فیزیکی حالت‌ها و ورودی‌ها را تعیین کن.

۷. Q, R اولیه را با Rule Bryson's بساز.

۸. LQR را محاسبه کن.

۹. $A - BK$ را بررسی کن.

۱۰. قطب‌های حلقه بسته را بررسی کن.

۱۱. پاسخ پله/پاسخ اولیه/trajectory را بررسی کن.

۱۲. میزان effort کنترلی را بررسی کن.

۱۳. saturation را شبیه‌سازی کن.

۱۴. noise را شبیه‌سازی کن.

۱۵. uncertainty را بررسی کن.

۱۶. tracking را بررسی کن.

۱۷. در صورت نیاز control LQI/LQG/MPC/robust را ارزیابی کن.

۳۲ فرمول‌های اصلی در یک نگاه

$$\dot{x} = Ax + Bu$$

$$u = -Kx$$

$$J = \int_0^\infty (x^T Q x + u^T R u) dt$$

$$A^T P + PA - PBR^{-1}B^T P + Q = 0$$

$$K = R^{-1}B^T P$$

$$A_{cl} = A - BK$$

$$\lambda_{cl} = \text{eig}(A - BK)$$

برای حالت گسسته:

$$x_{k+1} = A_d x_k + B_d u_k$$

$$K = (R + B^T P B)^{-1} B^T P A$$

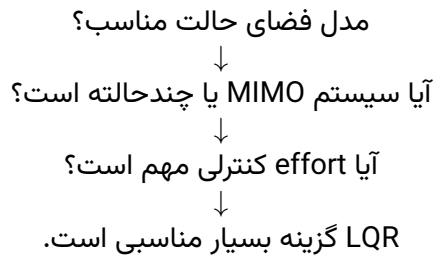
$$u_k = -K x_k$$

۳۳ توابع مهم MATLAB

کاربرد	تابع
طراحی LQR پیوسته	lqr
طراحی LQR گسسته	dlqr
حل CARE	care
حل DARE	dare
ماتریس کنترل‌پذیری	ctrb
ماتریس مشاهده‌پذیری	obsv
محاسبه مقادیر ویژه	eig
ساخت مدل فضای حالت	ss
پاسخ ناشی از شرایط اولیه	initial
پاسخ پله	step
شبیه‌سازی ورودی دلخواه	lsim
placement pole	place

۳۴ یک دید مهندسی برای انتخاب روش کنترل

می‌توان یک تصمیم‌گیری اولیه به صورت زیر داشت:



اگر محدودیت‌های سخت وجود داشته باشند:

LQR → MPC را نیز بررسی کن

اگر عدم قطعیت شدید باشد:

LQR → Control Robust را بررسی کن / H_∞

اگر حالت‌ها قابل اندازه‌گیری نباشند:

LQR → Observer/LQG

اگر tracking و error steady-state zero مهم باشد:

LQR → LQI یا ساختار tracking

اگر سیستم به شدت غیرخطی باشد:

LQR → TVLQR/iLQR/NMPC

۳۵ جمع‌بندی نهایی

LQR را می‌توان به صورت یک چارچوب بسیار منظم برای طراحی فیدبک حالت دید:

Model → Cost Define → Choose Q, R → Riccati Solve → K → Loop Closed Validate

قدرت اصلی LQR در این است که انتخاب بهره را از یک مسئله بهینه‌سازی مشخص استخراج می‌کند و هم‌زمان عملکرد حالت‌ها و تلاش کنترلی را در نظر می‌گیرد. اما برای یک پروژه صنعتی واقعی، طراحی K پایان کار نیست. باید حداقل موارد زیر نیز بررسی شوند:

Robustness + Tracking + Estimation State + Uncertainty Model + Delay + Noise + Saturation

در نتیجه، یک مهندس کنترل نباید صرفاً بداند چگونه

$$[K, S, e] = \text{lqr}(A, B, Q, R)$$

را اجرا کند؛ بلکه باید بتواند توضیح دهد:

۱. چرا LQR برای این سیستم انتخاب شده است؛

۲. چرا Q و R چنین مقادیری دارند؛

۳. آیا سیستم stabilizable است؛
۴. آیا حالت‌ها قابل اندازه‌گیری هستند؛
۵. آیا فرمان در محدوده عملگر است؛
۶. آیا مدل برای محدوده کاری معتبر است؛
۷. آیا tracking مورد نیاز است؛
۸. و آیا LQR در برابر عدم قطعیت‌های واقعی کافی است یا باید به MPC LQG، LQI یا Control Robust رفت.